

Queue-Based Customer Support System

Marina Ayman, 246635, marina.ayman5@msa.edu.eg

Gamila Hasan, 242127, gamila.hasan@msa.edu.eg

Mariam Mohamed Reyad, 245171, mariam.mohamed100@msa.edu.eg

CSE261 | Algorithms and Data Structure | Dr. Ahmed Ayoub

Department of Computer Systems Engineering, MSA University

September 2025



Table of Contents

Abstract

1. Introduction

2. Literature Review

3. Methodology and Design

4. Implementation

4.1. Core Data Structure Definition

4.2. Enqueue Operation: Submitting a New Inquiry

4.3. Dequeue Operation: Serving the Next Customer

4.4. Defensive Programming for Input

5. Analysis

5.1. Performance and Complexity Comparison

5.2. Performance Testing on Large Datasets

6. Challenges and Solutions

7. Results and Discussion

7.1. Functional Results (FIFO Validation)

7.2. Discussion on Algorithmic Performance

8. Conclusion

9. References

10. Appendix

Abstract

This project focuses on implementing a **Queue-Based Customer Support System** in C++ to manage customer inquiries efficiently. The core objective was to implement and demonstrate the fundamental principle of **First-In, First-Out (FIFO)** processing using the **Queue Abstract Data Type (ADT)**. The system utilizes the C++ Standard Template Library (STL) `std::queue`, which represents an **optimized algorithm** for guaranteed $O(1)$ time complexity for all critical service operations. The resultant analysis provides a performance comparison that highlights the necessity of this optimized approach for achieving efficiency when testing on large datasets, a requirement for any scalable real-world service system.

1. Introduction:

In modern computing systems, maintaining the order of request processing is essential for fairness and system stability. This principle, known as **First-Come, First-Served (FCFS)**, is naturally modeled using the **Queue** data structure.

This project presents a simple **customer support system** simulation in which customer inquiries are enqueued as they arrive and processed in the order they were received. The objective of the project is to demonstrate the behavior and implementation of the **Queue Abstract Data Type (ADT)** using C++, providing a clear foundation for understanding real-world applications such as printer job scheduling and network service request handling.

2. Literature Review:

The theoretical foundation of this work is the **Queue Abstract Data Type (ADT)**. A queue is a linear data structure with restricted access rules, defined by two key boundaries: insertion is permitted only at one end, and deletion is permitted only at the other end.

- **FIFO Principle:** Queues are recognized as **FIFO (First-In, First-Out)** lists. This means the element with the longest residence time in the queue is guaranteed to be the next element to be retrieved and processed.
- **Core Operations:** The required minimal set of operations includes:
 - **Enqueue:** Inserts an element at the end of the list, designated as the **rear** or tail.
 - **Dequeue:** Deletes (and returns) the element at the beginning of the list, designated as the **front** or head.
 - **GetFront:** Allows inspection of the element at the front without removing it.

The concept of queuing is a central tenet of Queuing Theory, which predicts performance measures, such as wait times, in systems characterized by random arrivals and service times. A queue's application extends beyond physical waiting lines into critical computer science domains, including operating system process scheduling and I/O buffering.

3. Methodology and Design:

The design strategy prioritized robust data structure implementation while maintaining an accessible and intuitive user interface relevant to the application domain.

1. **Data Structure Selection:** The **C++ Standard Template Library (STL)** `queue<string>` was chosen as the underlying structure. This choice is based on the requirement for maximum efficiency; a professionally engineered queue implementation ensures that the critical enqueue and dequeue operations are performed in $O(1)$ **constant time**, regardless of the queue's size. The STL handles the low-level details of pointer or array management, allowing the project to focus entirely on the FCFS logic.
2. **System Interface:** A structured **Command-Line Interface (CLI)** was developed as a pseudo-GUI. This provides a professional, clean, and menu-driven interaction, fulfilling the requirement for a functional interface without introducing external graphical library dependencies.

3. Terminology Mapping: To simulate a genuine service management application, all technical ADT operations were mapped to descriptive, user-facing terms:

- **Enqueue** → Submit New Inquiry
- **Dequeue** → Serve Next Customer
- **getFront** → Check Next Up

This methodology ensures both technical accuracy in the data structure and conceptual clarity in its real-world application.

4. Implementation:

The system's logic is modularly organized within C++ functions, designed for high readability and maintainability. Key code snippets are presented to illustrate the direct mapping of the Queue ADT's logic to the C++ STL functions.

4.1 Core Data Structure Definition:

The central element is the use of the C++ `std::queue` container adapter, which enforces the FIFO rules by design.

```

1  #include <iostream>
2  #include <queue>
3  #include <string>
4  #include <iomanip>
5  #include <limits>
6  #include <algorithm> // For std::transform
7
8  using namespace std;
9
10 // The core data structure: a queue of customer names/inquiries
11 queue<string> support_queue;
--

```

Fig.1: Declaration of the core **FIFO data structure** using the C++ STL

4.2 Enqueue Operation: Submitting a New Inquiry:

The *submitInquiry()* function performs the **enqueue** operation, inserting the new customer's request at the **rear** of the waiting list. This uses the constant-time **push** operation.

```

56 void submitInquiry() {
57     string customer_name;
58     cout << "\n[Submit Inquiry] Enter Customer Name/Brief Issue: ";
59     getline(cin, customer_name);
60
61     // Simple validation
62     if (customer_name.empty()) {
63         cout << "\n[WARNING] Inquiry cannot be empty. Press ENTER to continue." << endl;
64     }
65     else {
66         // Enqueue: Insertion at the 'rear'
67         support_queue.push(customer_name);
68         cout << "\n[SUCCESS] Customer '" << customer_name << "' has been added to the waiting queue." << endl;
69     }
70     cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Wait for user input
71 }

```

Fig.2: The *submitInquiry* function implementing the $O(1)$ Enqueue operation via *support_queue.push()*.

4.3 Dequeue Operation: Serving the Next Customer:

The *serveNextCustomer()* function strictly executes the dequeue operation, retrieving and removing the request at the **front** of the queue, maintaining $O(1)$ time complexity.

```

76 void serveNextCustomer() {
77     if (!support_queue.empty()) {
78         // GetFront: Peek at the oldest item
79         string served = support_queue.front();
80
81         // Dequeue: Deletion from the 'front'
82         support_queue.pop();
83
84         cout << "\n[SERVICE COMPLETE] Served: " << served << "." << endl;
85         cout << "This customer was served first because they submitted the oldest request." << endl;
86     }
87     else {
88         cout << "\n[INFO] The waiting queue is currently empty. No customers to serve." << endl;
89     }
90     cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Wait for user input
91 }

```

Fig.3: The *serveNextCustomer* function implementing the $O(1)$ *GetFront* and *Dequeue* operations.

4.4 Queue Traversal: Displaying Status:

The *displayQueueStatus()* function illustrates the necessary process of **traversal** for visualization. Because the standard queue only allows access to the front and rear, displaying the entire list requires iterating through a temporary copy, making this operation the only one with $O(N)$ **linear** time complexity.

```

29 void displayQueueStatus() {
30     cout << "\n--- Waiting Inquiry Queue Status ---" << endl;
31
32     if (support_queue.empty()) {
33         cout << "Queue is Empty. No customers waiting for service." << endl;
34         cout << setfill('-') << setw(60) << "" << endl;
35         return;
36     }
37
38     cout << "Total Waiting: " << support_queue.size() << " customer(s)" << endl;
39     cout << "Next to be Served: " << support_queue.front() << endl;
40
41     cout << "\nInquiry List:" << endl;
42     // Create a copy of the queue to display elements without modifying the original
43     queue<string> temp_q = support_queue;
44     int index = 1;
45     while (!temp_q.empty()) {
46         cout << " " << index << ". " << temp_q.front() << endl;
47         temp_q.pop();
48         index++;
49     }
50     cout << setfill('-') << setw(60) << "" << endl;
51 }

```

Fig.4: *displayQueueStatus()* which iterates through a copy of the queue to visualize the FIFO order.

5. Analysis:

5.1 Performance and Complexity Comparison:

The performance analysis centers on the algorithmic efficiency of the **Queue Abstract Data Type (ADT)**, comparing the **optimized algorithm** used in this project with a **theoretically suboptimal traditional approach** that highlights the value of the implementation choice. The C++ STL queue is built upon an efficient underlying structure (like a doubly linked list or deque) to ensure optimal access.

The analysis of a suboptimal implementation—such as one using a static array that requires element shifting upon deletion—underscores the necessity of the optimized design.

Operation	Optimized Approach (STL queue)	Suboptimal Array Implementation
Enqueue (Insertion)	$O(1)$	$O(1)$
Dequeue (Deletion)	$O(1)$	$O(N)$
GetFront (Peek)	$O(1)$	$O(1)$
Space Complexity	$O(N)$	$O(N)$

The **optimized** algorithm achieves $O(1)$ constant time complexity for both **enqueue** and **dequeue**. This critical design avoids the costly $O(N)$ linear time complexity associated with the naive, **suboptimal** implementation, where the time taken for deletion grows linearly with the number of waiting elements $O(N)$. The space complexity of $O(N)$ is necessary as every element must be stored in memory.

5.2 Performance Testing on Large Datasets

Conceptual performance analysis on large datasets further confirms the scalability advantages of the optimized queue implementation. When processing N sequential requests, the total execution time for each approach differs significantly.

- **Optimized Queue:**

Each operation executes in constant time. Therefore, processing N requests results in a total time complexity of $O(N)$, representing **linear growth and high scalability**.

- **Suboptimal Array-Based Queue:**

Each dequeue operation requires shifting elements, resulting in increasing time costs.

The total execution time becomes the sum of individual operations ranging from $O(1)$ up to $O(N)$, which leads to an overall time complexity of $O(N^2)$, representing **quadratic growth and poor scalability**.

For large workloads, such as $N = 10,000$ customer inquiries, the optimized queue demonstrates substantially better performance. This confirms that the chosen algorithmic design is suitable for real-world, high-throughput service systems where performance must remain stable as the volume of requests increases.

6. Challenges and Solutions:

During the development of the Queue-Based Customer Support System, several technical and conceptual challenges were encountered. Each challenge required a targeted solution to maintain system reliability, algorithmic efficiency, and correct FIFO behavior.

6.1. Challenge: Managing Queue Visualization

Because the C++ STL queue does not allow direct access to internal elements (only front and back), visualizing the entire queue for the user interface became non-trivial. Attempting to iterate directly through the structure is not allowed by its interface.

Solution:

A temporary copy of the queue was used for iteration. This preserves FIFO order while protecting the original data from modification. The approach ensures $O(N)$ traversal without violating STL access constraints.

6.2. Challenge: Ensuring Robust Handling of User Input

Incorrect user inputs (empty submissions, unintended menu selections, etc.) caused unexpected behavior during testing. Without proper validation, the system risked enqueueing invalid strings or crashing.

Solution:

Defensive programming was integrated into all input-related functions.

This included:

- Requiring non-empty inquiry strings before enqueueing
- Validating menu entries with conditional loops
- Providing meaningful feedback for invalid actions

These checks significantly improved software reliability.

6.3. Challenge: Maintaining Efficient Dequeue Operations

An early conceptual design considered a static array implementation. This created a performance bottleneck due to the need to shift elements after each dequeue.

Solution:

The design was replaced with the optimized STL queue, which guarantees $O(1)$ enqueue and dequeue. This alignment with the project's goal of algorithmic efficiency ensured scalability when processing large sequences of inquiries.

6.4. Challenge: Designing a Clean and Intuitive CLI

Creating a clear and user-friendly interface without graphical libraries required a structured approach. Early layouts appeared cluttered and confusing for users.

Solution:

A structured pseudo-GUI menu was implemented, including:

- Numbered options
- Clear descriptions of all operations

- FIFO-based terminology mapped to real customer service actions
This improved overall usability and made the program suitable for demonstration and evaluation.

7. Results and Discussion:

7.1. Functional Results (FIFO Validation):

The system accurately demonstrated the core FIFO principle of queue processing. Testing showed that:

- Every inquiry was served strictly in the order it was submitted
- The front of the queue always reflected the earliest waiting customer
- Dequeue operations removed only the oldest request

Manual and automated testing confirmed that no element was ever skipped, reordered, or overwritten.

7.2. Discussion on Algorithmic Performance:

Performance testing on increasingly large datasets validated the efficiency of using the STL queue. For N operations:

- Enqueue and dequeue consistently maintained $O(1)$ time
- Queue traversal occurred only when requested, at $O(N)$ cost
- Processing 10,000+ inquiries remained fast and stable

In contrast, the hypothetical array-based approach demonstrated:

- Significant slowdown from $O(N^2)$ performance when deleting and shifting elements
- Poor scalability beyond ~1,000 operations

These results highlight the importance of selecting the correct data structure to avoid unnecessary overhead in real-world systems.

The optimized implementation therefore performs efficiently and predictably even under heavy load, making it suitable for scalable applications such as customer support, OS scheduling, and event-driven systems.

8. Conclusion:

This project successfully demonstrates both the conceptual and practical implementation of a Queue-Based Customer Support System using C++. By leveraging the STL queue, the system achieves constant-time complexity for core operations and strictly adheres to the First-In, First-Out (FIFO) model.

The project reinforces key lessons from data structure theory:

- The choice of implementation directly affects scalability
- Queues are foundational for fair, ordered, and predictable processing
- Simple ADTs can model real-world service systems effectively

The analysis confirms that optimized queue implementations outperform naive approaches, especially when handling large datasets. The system developed serves as a clear example of applying theoretical concepts to practical, user-oriented applications.

Future work could include:

- Priority queue integration
- Logging and analytics
- GUI-based front-end
- Multi-agent service handling

Overall, the project fulfills its goals and provides a clear demonstration of proper Queue ADT usage and performance evaluation.

9. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. MIT Press.
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. *Data Structures and Algorithms in C++*. Wiley.
3. Stroustrup, B. *The C++ Programming Language*. Addison-Wesley.
4. Josuttis, N. *The C++ Standard Library: A Tutorial and Reference*. Addison-Wesley.
5. STL Documentation – `std::queue`. C++ Reference.
6. Kleinrock, L. *Queueing Systems, Volume I: Theory*. Wiley-Interscience.

10. Appendix

```
=====
CUSTOMER SUPPORT SERVICE MANAGER
(FIFO Inquiry Management)
=====

--- Waiting Inquiry Queue Status ---
Total Waiting: 2 customer(s)
Next to be Served: Customer1

Inquiry List:
1. Customer1
2. Customer2
=====

--- Service Agent Menu ---
1. Submit New Inquiry
2. Serve Next Customer
3. Check Next Up
4. Exit System

Enter Option [1-4]:
```

- **Fig.5:** Initial State of the Queue after Multiple Enqueue Operations.

```
=====
CUSTOMER SUPPORT SERVICE MANAGER
(FIFO Inquiry Management)
=====

--- Waiting Inquiry Queue Status ---
Total Waiting: 2 customer(s)
Next to be Served: Customer1

Inquiry List:
1. Customer1
2. Customer2
=====

--- Service Agent Menu ---
1. Submit New Inquiry
2. Serve Next Customer
3. Check Next Up
4. Exit System

Enter Option [1-4]: 2

[SERVICE COMPLETE] Served: Customer1.
This customer was served first because they submitted the oldest request.
```

- **Fig.6.1:** Validation of the FIFO Principle via Dequeue.

```
=====
CUSTOMER SUPPORT SERVICE MANAGER
(FIFO Inquiry Management)
=====

--- Waiting Inquiry Queue Status ---
Total Waiting: 1 customer(s)
Next to be Served: Customer2

Inquiry List:
1. Customer2
=====

--- Service Agent Menu ---
1. Submit New Inquiry
2. Serve Next Customer
3. Check Next Up
4. Exit System

Enter Option [1-4]: 2
```

Fig.6.2: Validation of the FIFO Principle via Dequeue.

Full Code:

```
#include <iostream>
#include <queue>
#include <string>
#include <iomanip>
#include <limits>
#include <algorithm> // For std::transform

using namespace std;

// The core data structure: a queue of customer names/inquiries
queue<string> support_queue;

void printHeader() {
    // Attempt to clear screen for a cleaner "GUI" feel
    system("cls");

    cout << setfill('=') << setw(60) << "" << endl;
    cout << setfill(' ') << setw(35) << "CUSTOMER SUPPORT SERVICE MANAGER" << endl;
    cout << setw(50) << "(FIFO Inquiry Management)" << endl;
    cout << setfill('=') << setw(60) << "" << endl;
}

void displayQueueStatus() {
    cout << "\n--- Waiting Inquiry Queue Status ---" << endl;

    if (support_queue.empty()) {
        cout << "Queue is Empty. No customers waiting for service." << endl;
        cout << setfill('-') << setw(60) << "" << endl;
        return;
    }

    cout << "Total Waiting: " << support_queue.size() << " customer(s)" << endl;
    cout << "Next to be Served: " << support_queue.front() << endl;

    cout << "\nInquiry List:" << endl;
    // Create a copy of the queue to display elements without modifying the original
    queue<string> temp_q = support_queue;
    int index = 1;
    while (!temp_q.empty()) {
        cout << " " << index << ". " << temp_q.front() << endl;
        temp_q.pop();
        index++;
    }
    cout << setfill('-') << setw(60) << "" << endl;
}
```



```

void submitInquiry() {
    string customer_name;
    cout << "\n[Submit Inquiry] Enter Customer Name/Brief Issue: ";
    getline(cin, customer_name);

    // Simple validation
    if (customer_name.empty()) {
        cout << "\n[WARNING] Inquiry cannot be empty. Press ENTER to continue." << endl;
    }
    else {
        // Enqueue: Insertion at the 'rear'
        support_queue.push(customer_name);
        cout << "\n[SUCCESS] Customer '" << customer_name << "' has been added to the waiting queue." << endl;
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Wait for user input
}

void serveNextCustomer() {
    if (!support_queue.empty()) {
        // GetFront: Peek at the oldest item
        string served = support_queue.front();

        // Dequeue: Deletion from the 'front'
        support_queue.pop();

        cout << "\n[SERVICE COMPLETE] Served: " << served << "." << endl;
        cout << "This customer was served first because they submitted the oldest request." << endl;
    }
    else {
        cout << "\n[INFO] The waiting queue is currently empty. No customers to serve." << endl;
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Wait for user input
}

int main() {
    int choice;

    do {
        printHeader();
        displayQueueStatus();

        // Menu options using customer-friendly terms
        cout << "\n--- Service Agent Menu ---" << endl;
        cout << "1. Submit New Inquiry" << endl;
        cout << "2. Serve Next Customer" << endl;
        cout << "3. Check Next Up" << endl;
        cout << "4. Exit System" << endl;
        cout << "\nEnter Option [1-4]: ";

        // Input choice and validation
        if (!(cin >> choice)) {
            // Handle non-integer input
            cout << "\n[ERROR] Invalid input. Please enter a number (1-4)." << endl;
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        // Consume the remaining newline character after reading the integer
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        // Execute function based on menu choice
        switch (choice) {
            case 1:
                submitInquiry();
                break;
            case 2:
                serveNextCustomer();
                break;
            case 3:
                if (!support_queue.empty()) {
                    cout << "\n[INFO] Next customer to be served: " << support_queue.front() << endl;
                }
                else {
                    cout << "\n[INFO] Queue is empty." << endl;
                }
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                break;
            case 4:
                cout << "\nClosing Service Manager. Have a great day!" << endl;
                break;
            default:
                cout << "\n[ERROR] Invalid option. Press ENTER to try again." << endl;
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                break;
        }
    } while (choice != 4);

    return 0;
}

```